Kasi Yedumati · Full Stack Training

Lesson 6 — CRUD

Course: FastAPI for Full Stack · **Session 6 of 10** · Learning path · **Previous:** Lesson 5 · **Next:** Lesson 7

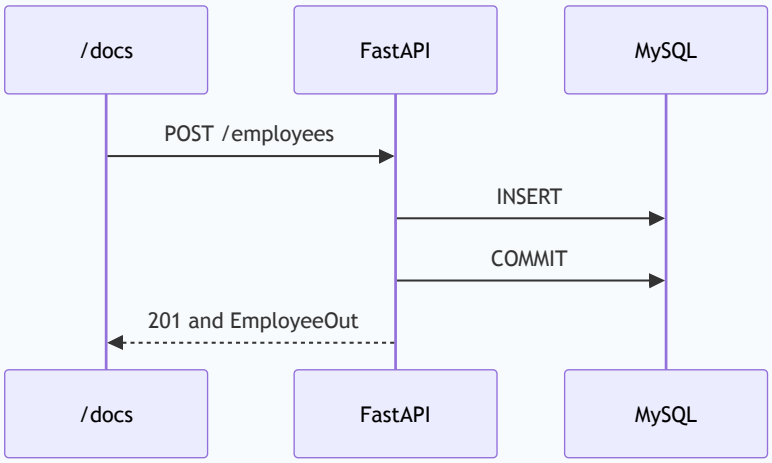
> **Module Outcome:** You can create, replace, update, and delete an employee in MySQL, with 201, 200, 204, 400, and 404.

You will build	The full employee API on the `employee` table
New words	Commit, rollback
You need	The `employee` table, a session, and `EmployeeOut`

From the requirements. HR registers an employee, corrects a designation, and removes a record. Those writes have to survive a restart, so they go through the session into MySQL.

The file that does all of it is `examples/lesson06/01_crud.py`. It reads the table the same way as the page before this one. The new piece is `commit` after a change, and `rollback` when MySQL rejects a duplicate id.

1. Create, then commit




```
from sqlalchemy.exc import IntegrityError

@app.post("/employees", status_code=201, response_model=EmployeeOut)
def create_employee(employee: EmployeeCreate):
    db = SessionLocal()
    try:
        row = Employee(**employee.model_dump())
        db.add(row)
        db.commit()
        db.refresh(row)
        return row
    except IntegrityError:
        db.rollback()
        raise HTTPException(status_code=400, detail="Employee id already exists")
    finally:
        db.close()
```

Line	Job
`db.add`	Stage the new row in the session. MySQL does not have it yet
`db.commit`	Write it
`db.refresh`	Read the new numeric `id` back onto the object
`IntegrityError`	MySQL refused the insert because `employee_id` is already there
`db.rollback`	Throw away the failed insert so the session can close cleanly

`EmployeeCreate` is the registration model. A bad email is still **422**, and `create_employee` does not run, so there is nothing to roll back.

Send Sana again:

```
{
  "employee_id": "E-1048",
  "name": "Sana Qureshi",
  "designation": "HR Executive",
  "department": "Human Resources",
  "email": "sana.qureshi@acme.example"
}
```

201. `SELECT * FROM employee WHERE employee_id = 'E-1048';` shows the row. Send the same body again. **400** and `Employee id already exists`.

---

2. Replace, patch, delete

PUT writes every field from `EmployeeCreate`. The id in the URL replaces whatever id was in the body.

```
for key, value in employee.model_dump().items():
    setattr(row, key, value)
row.employee_id = employee_id
db.commit()
db.refresh(row)
```

PATCH writes only `changes.model_dump(exclude_unset=True)`.

DELETE removes the row and returns **204** with an empty body:

```
db.delete(row)
db.commit()
return Response(status_code=204)
```

Each of these opens `SessionLocal()`, uses `try` / `finally` to close it, and raises **404** when the id is not in the table.

> **Common error:** You `add` and return the row without `commit`. `/docs` shows the new employee, you restart, and the row is gone.

Common error: After `IntegrityError` you do not `rollback`. The next request on that session raises a second error about a failed transaction. This function creates a new session each time, and `rollback` keeps that session honest before `close`.

Try these

1. **(Easy)** POST `E-1048`. Restart `fastapi dev`. GET `E-1048`. Sana is still there.
2. **(Easy)** PATCH only her designation. In MySQL, `email` is unchanged.
3. **(Medium)** DELETE `E-1048`. The response is **204** and empty. The `SELECT` returns no row.
4. **(Medium)** PUT `E-1042` with a bad email. Status **422**. Ananya's row in MySQL is unchanged.
5. **(Stretch)** POST an id that exists. Name the exception MySQL raised and the status you returned.

Interview: *What does commit do?* — It tells MySQL to keep the staged insert, update, or delete. Until commit, a restart of the API loses that change.

Key takeaways

- A write is `add` or `delete`, then `commit`, then `refresh` when you need the row back.
- A duplicate `employee_id` is an `IntegrityError`. Roll back and return **400**.
- A bad body is still **422** and never reaches `commit`.
- GET, PUT, PATCH, and DELETE use the HR id `E-1042`.

Lab — Ticket ACME-206

Homework

Next: Project structure. The same routes move into separate files, and `Depends` hands each route a session.